МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №X
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Т.В. Бессонов
Группа: М8О-307БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы. Реализовать камеру с возможностью перемещения и освещение по модели Блинна–Фонга с использованием нескольких типов источников света.

Вариант: 2 — октаэдр (объект из предыдущей лабораторной работы).

Выполнено. Базовое задание и дополнительные задания 1–3:

Метод решения

Математическая часть

Камера. Положение камеры задаётся вектором p, ориентация — углами Эйлера. Матрица вида строится как $V = R^T \cdot T(-p)$, где `R` — матрица поворота камеры, а `T` — матрица переноса. Направление взгляда вычисляется как d = R · (0, 0, −1). Перспективная проекция строится функцией glm::perspectiveFov по вертикальному углу обзора FOV, отношению сторон окна и плоскостям отсечения near = 0.1, far = 100. Дополнительно сохранена возможность переключения на ортографическую проекцию glm::orthoZO.

Модель освещения Блинна–Фонга. Для каждого фрагмента вычисляются:

- мировое положение p_world, переданное из вершинного шейдера;
- нормаль *N = normalize(M · n_local)*, M – матрица модели;
- направление на наблюдателя *V = normalize(camera_position − p_world);*
- направление на источник L;
- половинный вектор *H = normalize(L + V).*

Вклад источника:

diffuse = k_d · max(N · L, 0) · C_light · I
specular = k_s · max(N · H, 0)^shininess · C_light · I
где k_d — альбедо материала, k_s — цвет/коэффициент блика, shininess — степень зеркальности, C_light — цвет источника, I — его интенсивность.
Рассеянный свет: *ambient = C_amb · k_amb.*
Точечный источник. L = normalize(light_position − p_world), затухание:
$attenuation = 1 / (1 + k_{linear} \cdot d + k_{quadratic} \cdot d^2)$
где d — расстояние до источника. Квадратичный член обеспечивает затухание по закону обратных квадратов.
Прожектор. Дополнительно вычисляется косинус угла между направлением прожектора и направлением от источника на фрагмент:
cos_angle = dot(−L, normalize(spot_direction))
Если cos_angle < cos(outer_angle), фрагмент не освещается. Мягкий край реализован коэффициентом:
smooth = clamp((cos_angle − cos_outer) / (cos_inner − cos_outer), 0, 1)

Внутренний и внешний углы хранятся в градусах, а перед загрузкой в буфер переводятся в косинусы. Итоговый вклад прожектора умножается на `smooth` и на затухание.

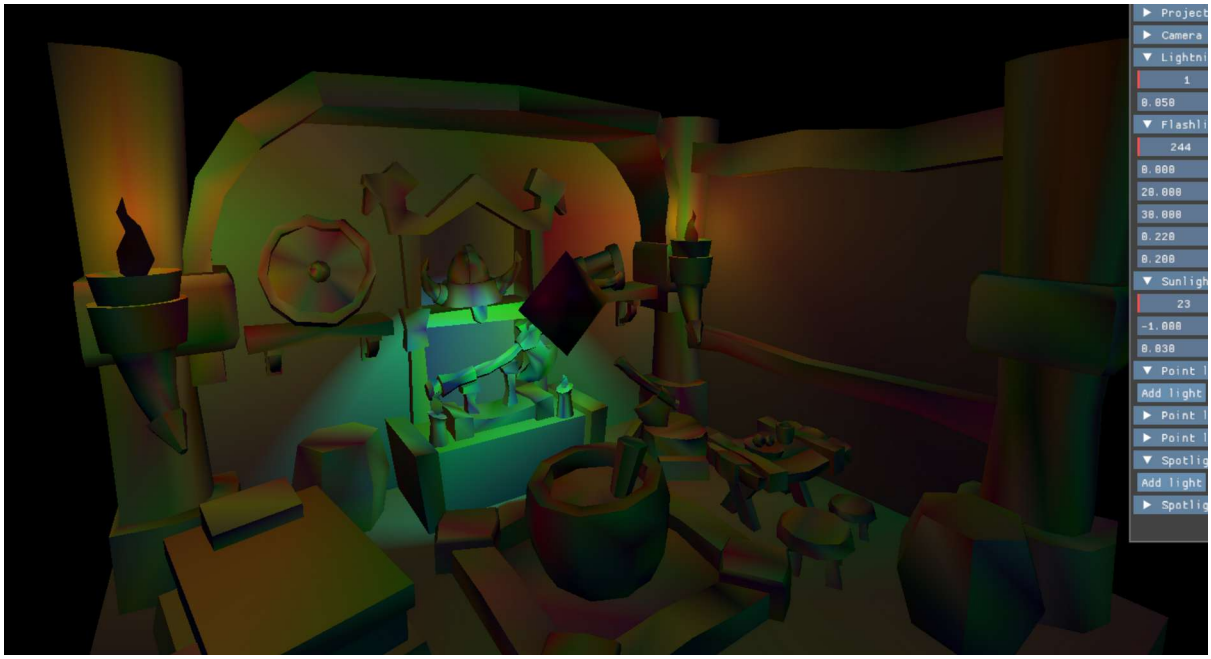
Генерация нормалей. Для октаэдра нормали вычисляются процедурно: для каждого треугольника находится вектор *n = cross(p2 − p1, p3 − p1)*, который прибавляется к нормалям всех трёх вершин; после обхода всех граней нормали нормализуются. Для загружаемой OBJ-модели используются нормали из файла.

Хранение источников света. Точечные источники и прожекторы хранятся в отдельных storage-буферах с массивами переменной длины (`std430`). В шейдере длина массива не фиксируется: количество активных источников передаётся в uniform-буфере сцены (point_light_count, spotlight_count).

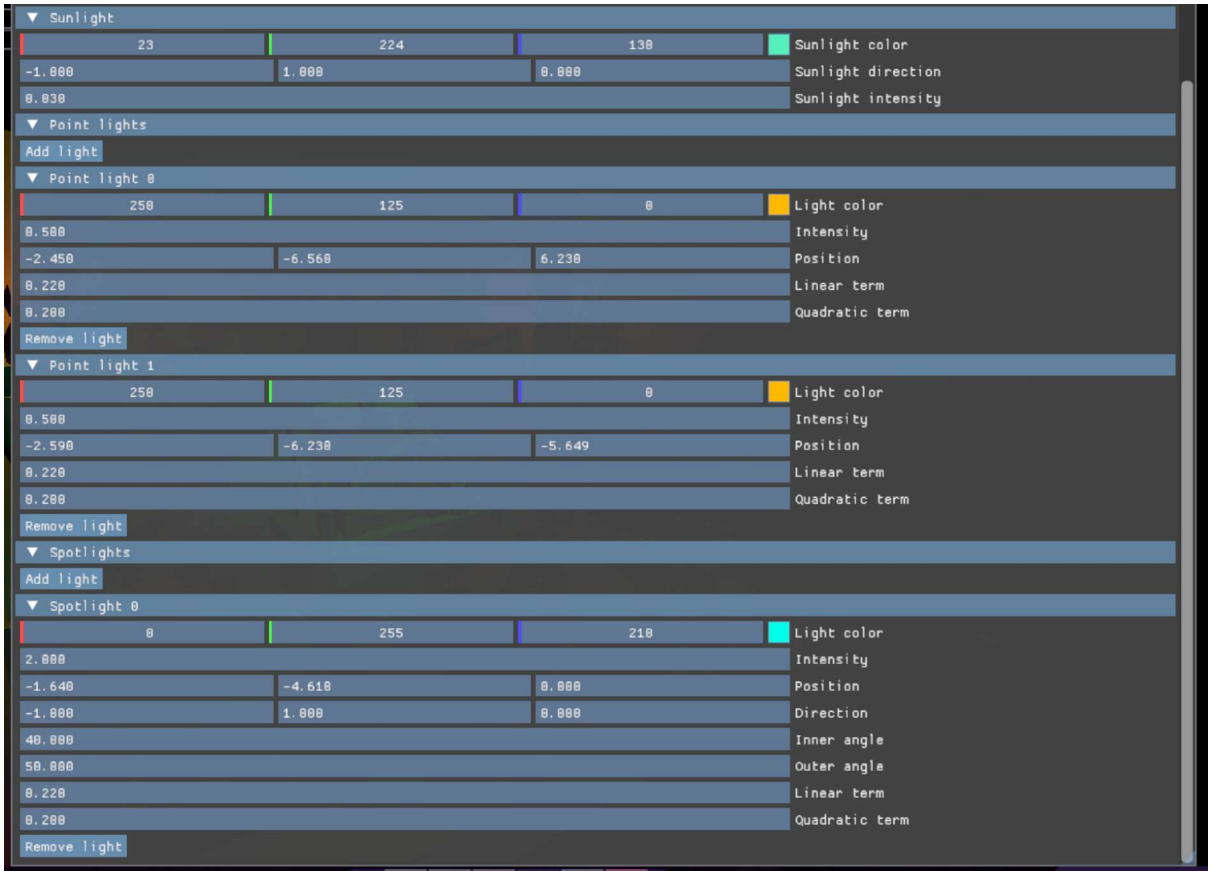
Объекты Vulkan

- **Графический конвейер:** вершинный и фрагментный шейдеры GLSL 450; входные атрибуты вершины — позиция (location 0), цвет (location 1) и нормаль (location 2); топология `TRIANGLE_LIST`;
- **Набор дескрипторов сцены (set 0):**
  - binding 0 — uniform-буфер `SceneUniform`: матрица `projection_view`, параметры `SceneLightning` (фонарик, положение камеры, ambient, направленный свет, количества источников);
  - binding 1 — storage-буфер с массивом точечных источников;
  - binding 2 — storage-буфер с массивом прожекторов.
- **Набор дескрипторов модели (set 1):** binding 0 — uniform-буфер `FigureUniform` с матрицей модели, основным цветом и параметрами материала (`albedo`, `specular_strength`, `shininess`). Для каждой фигуры выделяется отдельный набор дескрипторов, который привязывается перед её отрисовкой.
- **Пул дескрипторов** создаётся с учётом количества фигур и двух storage-буферов; наборы обновляются при инициализации и при изменении привязок.
- **Буферы** создаются через Vulkan Memory Allocator с отображением в память хоста; структуры выровнены под `std140` для uniform-буферов и `std430` для storage-буферов.

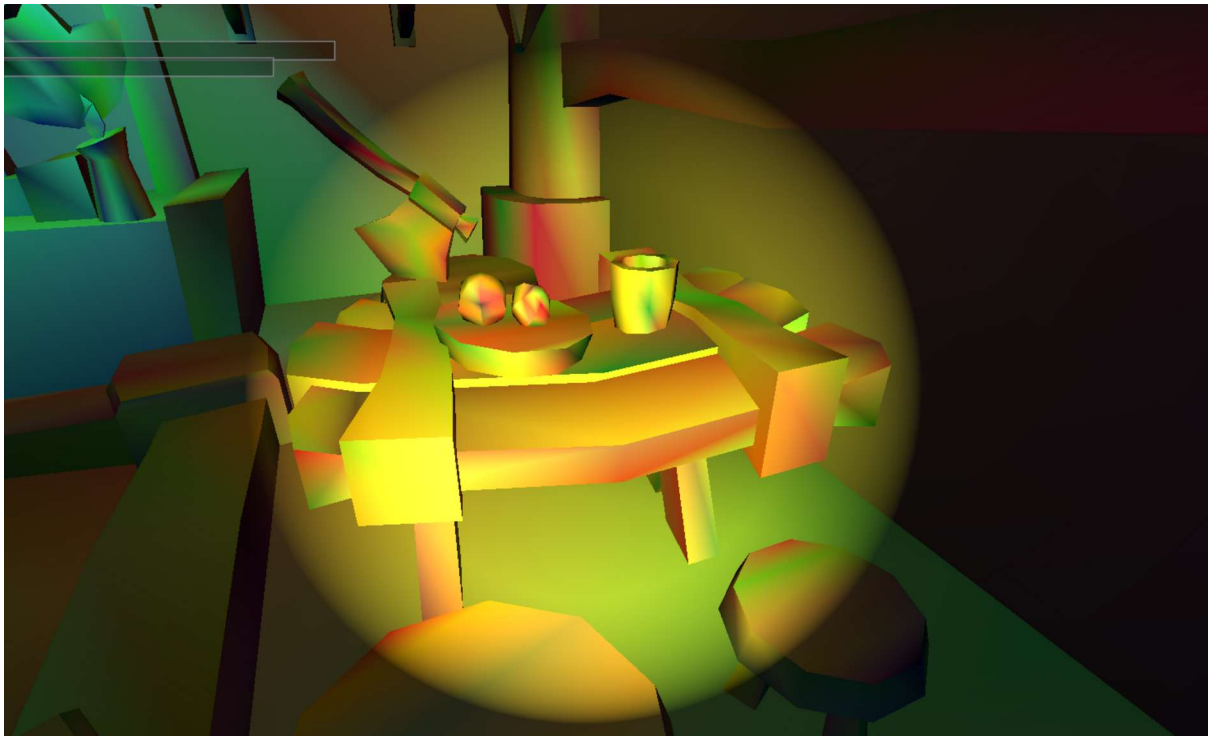
Результаты



Скриншот 1. Сцена с октаэдром и моделью `viking_room.obj`. Видны вклад направленного света, рассеянного света и двух точечных источников оранжевого цвета. Топор освещен прожекторным светом.



Скриншот 2. Конфигурация освещения на сцене.



Скриншот 3. Фонарик светит на стол.

Выводы

Я научился реализовывать камеру с перемещением и вращением, а также освещать сцену по модели Блинна–Фонга с направленным, точечными и прожекторными источниками света. Я освоил хранение массивов источников света в storage-буферах, реализацию затухания по закону обратных квадратов и мягких краёв прожектора. Кроме того, я научился процедурно генерировать нормали вершин и управлять всеми параметрами сцены, источников и материалов в реальном времени через интерфейс ImGui. Я научился загружать модели в формате obj через библиотеку tinyobjloader.